

RELATÓRIO TÉCNICO

PROJETO AGROPEC. ANÁLISE DE QUALIDADE E SEGURANÇA

Evidências da análise realizada no SonarQube
AEP 6S — 1ª Entrega

1. Objetivo

Este relatório tem como objetivo documentar as atividades realizadas no projeto AgroPec, com foco na organização da API, implementação das funcionalidades de gerenciamento dos animais, persistência dos dados, validação das entradas e tratamento de apontamentos relacionados à qualidade e segurança do código. Os prints do SonarQube são utilizados como evidências visuais dos resultados obtidos durante as análises.

2. Projeto analisado

O AgroPec é uma aplicação desenvolvida em Java com Spring Boot, estruturada como uma API REST para gerenciamento de informações de animais. O projeto utiliza MongoDB para persistência e organiza o código em camadas, separando responsabilidades entre controller, service, repository, DTO e model.

3. Atividades realizadas

3.1 Organização da API e CRUD

Foi estruturada a API para realizar as operações de cadastro, consulta, atualização e exclusão de animais. O AnimalController recebe as requisições HTTP, o AnimalService concentra a lógica de serviço e o AnimalRepository realiza o acesso aos dados.

- Cadastro de animais.
- Consulta de animais.
- Atualização de registros.
- Exclusão de registros.
- Listagem dos animais cadastrados.

3.2 Persistência dos dados

A persistência foi implementada utilizando Spring Data MongoDB. O AnimalModel representa o documento armazenado e o AnimalRepository utiliza a infraestrutura do Spring Data para executar as operações sobre o banco.

3.3 Validação dos dados

Foi utilizado um DTO para receber os dados enviados nas requisições. As informações de entrada possuem validações, evitando que dados obrigatórios ou valores fora das regras definidas sejam aceitos pela aplicação.

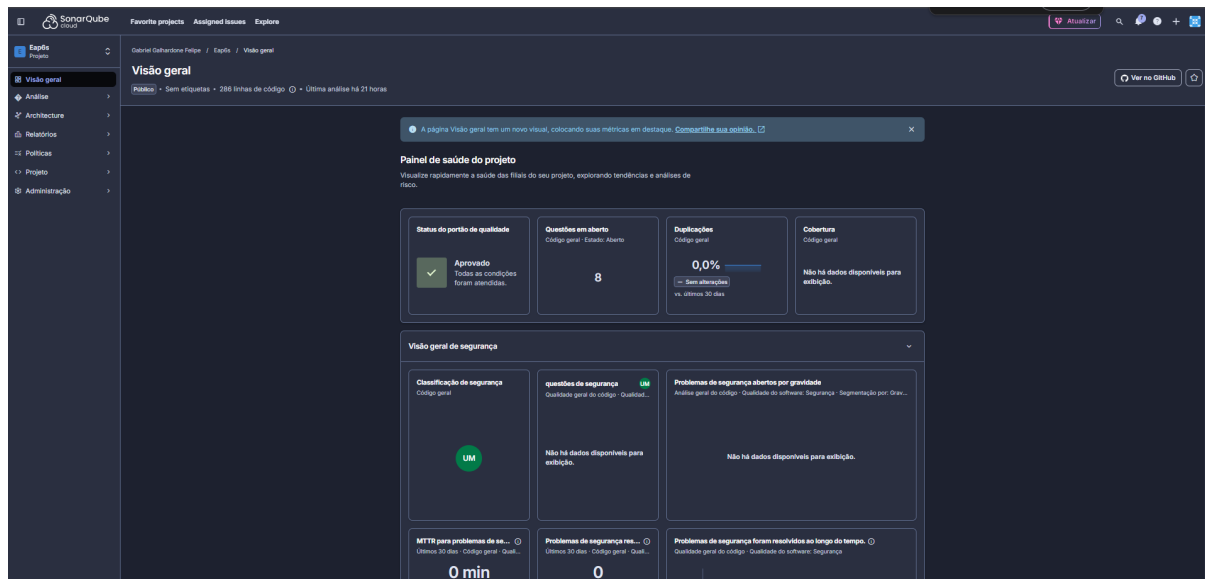
3.4 Segurança e configuração

Durante a evolução do projeto foram adotadas medidas para evitar que configurações sensíveis fiquem diretamente expostas no código versionado. A aplicação utiliza configuração externa por meio de variáveis/arquivo .env, e o arquivo .env está incluído no .gitignore.

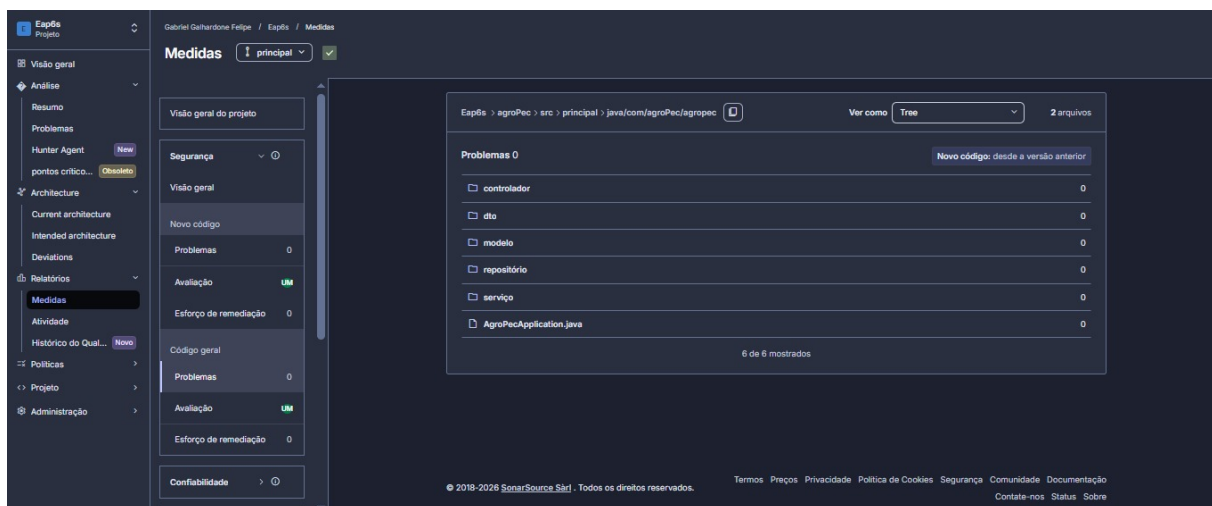
Essa abordagem é importante porque permite manter informações de configuração fora do código fonte, reduzindo o risco de exposição de credenciais ou outros valores sensíveis.

4. Análise de qualidade realizada no SonarQube

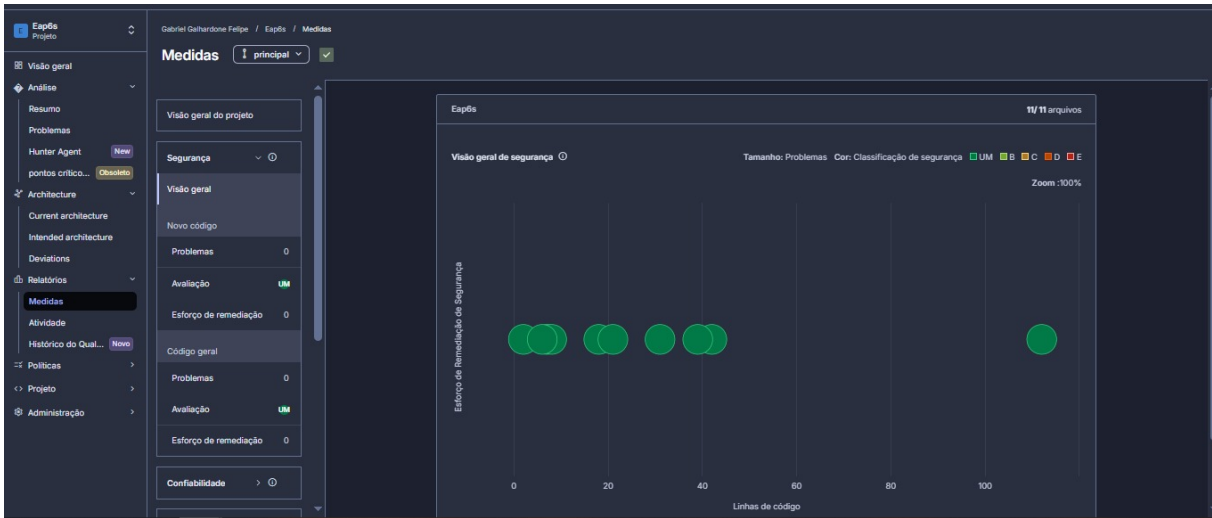
Os prints abaixo registram diferentes visões do projeto dentro do SonarQube. Eles servem como evidência do acompanhamento da qualidade e segurança do código durante o desenvolvimento.



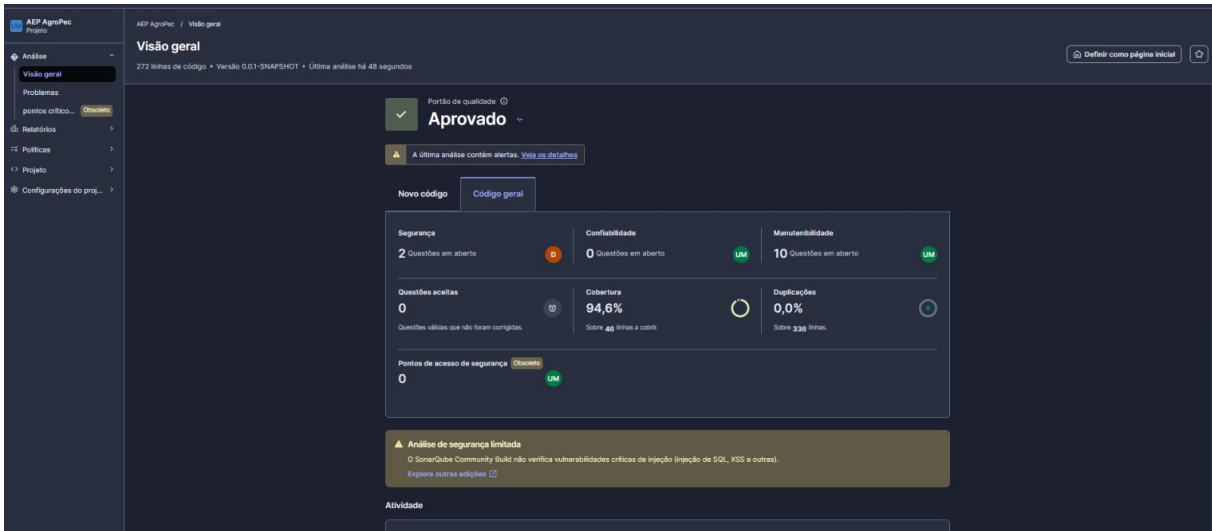
Na primeira evidência é possível observar o painel de saúde do projeto. O SonarQube apresenta o status do Quality Gate como aprovado, além dos indicadores relacionados a questões abertas, duplicações e cobertura.



A segunda evidência mostra a estrutura do código analisado. É possível identificar a separação das principais camadas da aplicação: controller, dto, model, repository e service, além da classe principal AgroPecApplication.java. A organização por camadas facilita a manutenção e a evolução do sistema.



A terceira evidência apresenta a visão gráfica das métricas de segurança. O gráfico relaciona as linhas de código dos arquivos analisados com o esforço de remediação indicado pela ferramenta.



A quarta evidência é a principal referência para os indicadores apresentados neste relatório. O painel informa que o Quality Gate está aprovado. Também são apresentados 2 apontamentos de segurança, que após ajustes no código saíram conforme os prints acima. 0 questões de confiabilidade, 10 questões de manutenibilidade, cobertura de 94,6% e 0,0% de duplicações.

5. Resultados apresentados no SonarQube

Indicador	Resultado no print	Descrição
Quality Gate	Aprovado	As condições do portão de qualidade foram atendidas.
Segurança	0	Não há questões de Segurança.
Confiabilidade	0	Não há questões de confiabilidade registradas.

Manutenibilidade	0	Não há questões de manutenibilidade registradas
Cobertura	94,6%	Percentual de cobertura indicado pela análise.
Duplicações	0,0%	Não foram identificadas duplicações no resultado apresentado.
Pontos de acesso de segurança	0	Nenhum ponto de acesso de segurança foi registrado na visão apresentada.

6. Tratamento das questões de segurança

As duas questões de segurança que aparecem no print foram tratadas durante a implementação. A principal preocupação foi evitar que informações de configuração sensíveis fossem mantidas diretamente no código-fonte versionado.

Para isso, a configuração foi preparada para ser fornecida externamente e o arquivo `.env` foi adicionado ao `.gitignore`. Dessa forma, informações que possam conter credenciais ou parâmetros de ambiente não precisam ser armazenadas no repositório.

É importante destacar que o print representa o estado da análise no momento em que foi gerado. Por isso, mesmo com a correção realizada no código, o painel pode continuar mostrando as duas questões até que uma nova análise seja executada no SonarQube.

7. Evidências no código-fonte

Arquivo / componente	Atividade relacionada
<code>AnimalController.java</code>	Implementação dos endpoints da API.
<code>AnimalService.java</code>	Camada responsável pela lógica de serviço.
<code>AnimalRepository.java</code>	Acesso e persistência dos dados no MongoDB.
<code>AnimalModel.java</code>	Representação do documento de animal.
<code>AnimalRequestDTO.java</code>	Recebimento e validação dos dados de entrada.
<code>AgroPecApplication.java</code>	Inicialização da aplicação e carregamento de configuração.
<code>.gitignore</code>	Prevenção do versionamento do arquivo <code>.env</code> .
<code>application.properties</code>	Configurações da aplicação e conexão com o ambiente.

8. Conclusão

A análise realizada demonstra que o projeto AgroPec passou por uma etapa de organização e melhoria da qualidade do código. Foram estruturadas as camadas da aplicação, implementadas operações de gerenciamento de animais, configurada a persistência em MongoDB e adicionadas validações para os dados recebidos pela API.

Na parte de segurança, foram realizadas alterações para retirar configurações sensíveis do código versionado, utilizando configuração externa e protegendo o arquivo .env por meio do .gitignore. Essas alterações correspondem ao tratamento das questões de segurança identificadas durante o desenvolvimento.

Os prints também demonstram que o Quality Gate aparece como aprovado, com 94,6% de cobertura e 0,0% de duplicações na análise apresentada. Como o painel ainda exibe duas questões de segurança em aberto, recomenda-se executar uma nova análise após as correções para atualizar oficialmente o status dessas questões no SonarQube.

CASOS DE TESTE —TESTE FUNCIONAL — AGROPEC

9.DADOS UTILIZADOS NOS TESTES

POST: brinco TEST-001, pesoEntrada 450, raçaNelore. PUT: brinco TEST-001-UPD, pesoEntrada 460, raça Nelore. O ID usado no fluxo é armazenado na variável animalId da coleção Postman.

10.CASOS DE TESTE

Caso	Teste	O que verifica
CT-01	GET /animais/list	Listar animais
CT-02	POST /animais/create	Cadastrar animal
CT-03	GET /animais/list/{id}	Buscar animal por ID
CT-04	PUT /animais/update/{id}	Atualizar animal
CT-05	DELETE /animais/delete/{id}	Excluir animal

11.CONCLUSÃO

Os endpoints estão implementados no AnimalController com os métodos HTTP e códigos de sucesso esperados. Para registrar aprovação definitiva no relatório acadêmico, é necessário executar a aplicação localmente com MongoDB ativo e rodar a coleção no Postman;

